



Figure 5: Kernel configuration using the graphical genkernel tool

4. To minimise the build noise, you can use the following command:

```
make > /dev/null
```

This will just direct the output from **make** (but you can still see the errors). I would suggest that you redirect them to **/dev/null**.

5. **An advanced feature:** The **make** program allows you to split the entire build process into a number of **jobs**. So, if your processor is quite efficient, you can split the process and run the jobs concurrently. This is a recommended way for hackers, as this will reduce your I/O wait time. You can just use the command given below:

```
make -jn
```

...where **n** is the number of jobs. You may choose this (value of **n**) based on your processor.

When it comes to the kernel installation side, it is better that you follow your architecture and boot loader specific instructions. But for modules you can simply issue the command below as the root:

```
make modules_install
```

6. Novices can use the files like the **package-list**, which may be located in the root of the source (for example, **/linux-source-kernel_version/package-list**) to comprehend the files included, dependency (kernel image and modules), etc.

Package: acpi-modules

Depends: kernel-image

Priority: standard

Description: Support for ACPI

Package: fat-modules

Depends: kernel-image

Priority: standard

Description: FAT filesystem support

This includes Windows FAT and VFAT support.

Package: fb-modules

Depends: kernel-image

Priority: standard

Description: Framebuffer modules

Back to action

When you are programming, you can find that the kernel is not linked to the standard C library, mostly due to the size and speed factors (as even a **decent subset** of the C library is quite large for the kernel). But you can use many **libc** functions (say, functions for string manipulation) as they are implemented inside the kernel.

Now, you might ask about the **printf()** in C. Of course, the kernel can't access **printf()** but it has access to **printk()**. It uses the **syslog** program to read the log buffer entries made by **printk()** and its (function's) usage is similar to that of **printf()**.

Process descriptor and the task structure

Task list (a circular doubly-linked list) is used by the kernel to record the list of processes.

Each element is a process descriptor of type struct **task_struct** and the reference can be found in **/linux/sched.h**:

```
extern void proc_sched_show_task(struct task_struct *p, struct seq_file *m);
extern void proc_sched_set_task(struct task_struct *p);
extern void
print_cfs_rq(struct seq_file *m, int cpu, struct cfs_rq *cfs_rq);
#else
static inline void
proc_sched_show_task(struct task_struct *p, struct seq_file *m)
{
}
static inline void proc_sched_set_task(struct task_struct *p)
{
}
static inline void
print_cfs_rq(struct seq_file *m, int cpu, struct cfs_rq *cfs_rq)
{
}
#endif
```

```
extern unsigned long long time_sync_thresh;
```

The process descriptor has all the data concerning a specific process. The **task_struct** structure is allocated using the slab allocator (to enable cache colouring and object reuse). If you look at Figure 6, you can see the struct **thread_info**. It remains on the top and bottom of it, for the stacks that grow up. The following code shows the **thread_info** structure defined on an **x86** as:

```
struct thread_info {
    struct task_struct *task;
```